TypeScript: Manual Completo de Arquitetura & POO

Documentação de Referência Extensiva — Tipagem Estrutural, Encapsulamento Avançado, Polimorfismo e Design Patterns

Capítulo 1: O Ecossistema TypeScript e Tipagem Estática

1.1 O que é TypeScript e Transpilação

> **Definição Formal:** TypeScript é uma linguagem de programação de código aberto desenvolvida pela Microsoft que atua como um *superset* (supersistema) tipado do JavaScript. Ela adiciona checagem estática de tipos em tempo de compilação sem alterar o comportamento de execução do JavaScript final.

Diferente de linguagens como C# ou Java que compilam diretamente para bytecode ou código de máquina, o compilador do TypeScript (`tsc`) realiza um processo chamado **Transpilação** (compilação de código-fonte para código-fonte). O processo remove todos os anotações de tipo, interfaces e decoradores, gerando JavaScript puro (ES5/ES6+) pronto para ser executado em navegadores ou no Node.js.

1.2 Tipagem Estática vs Tipagem Dinâmica

> **Definição:** Em sistemas dinâmicos (JavaScript), a validação dos tipos ocorre durante a execução (Runtime), podendo causar erros fatais. Em sistemas estáticos (TypeScript), a verificação ocorre no tempo de compilação (Compile-time), garantindo correção conceitual antes que o código chegue ao usuário.

```
// JavaScript Tradicional (Erro silencioso em runtime)
function calcular(a, b) { return a.toFixed(2) + b.toFixed(2); }
// Se passar uma string, quebra o sistema em tempo de execução!

// TypeScript (Validação em tempo de compilação)
function calcularSeguro(a: number, b: number): string {
    return (a + b).toFixed(2);
}
```

1.3 Tipagem Estrutural vs Tipagem Nominal

> **Definição:** O TypeScript utiliza o modelo de *Sistema de Tipos Estrutural* (Duck Typing). A compatibilidade de dois tipos não é determinada pelo nome de suas classes/interfaces ou sua declaração explicita, mas exclusivamente pelo formato e forma de suas estruturas membros.

Se um objeto possui todas as propriedades e métodos exigidos por uma interface, o TypeScript o considera válido, independentemente da hierarquia formal de herança.

Capítulo 2: Sistema de Tipos Primitivos e Especiais

2.1 Tipos Primitivos da Linguagem

> **Definição:** Tipos primitivos representam os valores fundamentais imutáveis manipulados pelo computador. No TypeScript englobam `string`, `number`, `boolean`, `symbol`, `bigint`, `null` e `undefined`.

```typescript
// Declarações explícitas com imutabilidade e validação
const nomeEmpresa: string = "Tech Solutions";
const faturamentoAnual: number = 1500000.50;
const emAtividade: boolean = true;
const idUnico: symbol = Symbol("id");
const grandeNumero: bigint = 9007199254740991n;
```

2.2 Tipos Especiais: any, unknown, void, never

> **Definição dos Tipos Especiais:**
>
> - `any`: Desativa completamente a checagem de tipos (Escape-hatch). Deve ser evitado.
> - `unknown`: O oposto seguro do `any`. Representa qualquer valor, mas exige checagem de tipo (Type Guard) antes de ser utilizado.
> - `void`: Indica explicitamente a ausência de retorno em uma função.
> - `never`: Representa valores que nunca ocorrem (ex: funções que lançam exceções irrecuperáveis ou loops infinitos).

```typescript
function processarEntradaDesconhecida(dado: unknown): void {
    // Exige Type Guard para garantir segurança estática
    if (typeof dado === "string") {
        console.log(dado.toUpperCase());
    }
}

function falharSistema(msg: string): never {
    throw new Error(`[ERRO FATAL]: ${msg}`);
}
```

2.3 Checagem de Null e Undefined (Strict Null Checks)

Quando a opção `strictNullChecks` está ativada no `tsconfig.json`, os valores `null` e `undefined` não pertencem ao domínio de outros tipos. Uma variável do tipo `string` não pode aceitar `null` sem o uso explícito de Union Types.

Capítulo 3: Coleções Estruturadas e Tipagem Metódica

3.1 Arrays e ReadonlyArrays

> **Definição:** Estruturas de dados homogêneas que armazenam sequências de elementos. O tipo `ReadonlyArray<T>` cria coleções completamente imutáveis que impedem métodos modificadores como `push()`, `pop()` ou atribuição direta de índices.

```
// Array Modificável
const listaProdutos: string[] = ["Monitor", "Teclado"];
listaProdutos.push("Mouse");

// Array Imutável (Garantia de Segurança)
const codigosFixos: ReadonlyArray<number> = [101, 202, 303];
// codigosFixos.push(404); // Erro de compilação!
```

3.2 Tuplas Imutáveis e Desestruturação

> **Definição:** Uma Tupla é um tipo heterogêneo fixo onde a quantidade exata de elementos e os tipos de cada posição individual são estritamente pré-definidos na declaração.

```
// Declaração de Tupla: [Latitude, Longitude, NomeLocal]
type CoordenadaGPS = [number, number, string];

const pontoPernambuco: CoordenadaGPS = [-8.0476, -34.8770, "Recife"];

// Desestruturação fortemente tipada
const [lat, lng, cidade] = pontoPernambuco;
```

3.3 Enums Numéricos e Enums de String

> **Definição:** Estruturas nomeadas para agrupamento de constantes relacionadas. Enums de String não sofrem problemas de mapeamento reverso involuntário e são mais recomendados para legibilidade e depuração.

```
enum PapelUsuario {
    ADMIN = "ADMINISTRADOR",
    EDITOR = "EDITOR_CONTEUDO",
    CLIENTE = "CLIENTE_FINAL"
}
```

Capítulo 4: Paradigma Orientado a Objetos — Estrutura de Classes

4.1 Anatomia de uma Classe no TypeScript

> **Definição:** Uma Classe é um modelo estrutural (blueprint) que descreve o estado (atributos/propriedades) e o comportamento (métodos) de objetos concretos instanciados em memória através do operador `new`.

```
class Veiculo {
    // Declaração de atributos com tipagem e inicialização
    marca: string;
    modelo: string;
    ano: number;

    constructor(marca: string, modelo: string, ano: number) {
        this.marca = marca;
        this.modelo = modelo;
        this.ano = ano;
    }

    exibirFichaTecnica(): string {
        return `${this.marca} ${this.modelo} - Ano: ${this.ano}`;
    }
}
```

4.2 Parameter Properties (Propriedades via Construtor)

> **Definição:** Um recurso sintático exclusivo do TypeScript que permite declarar e inicializar atributos da classe diretamente na assinatura do construtor, eliminando atribuições manuais redundantes.

```
// Sintaxe Concisa equivalente à classe completa anterior
class VeiculoSintetico {
    constructor(
        public marca: string,
        public modelo: string,
        public ano: number
    ) {}
}
```

4.3 Atributos Readonly em Objetos

A palavra-chave `readonly` assegura que determinado atributo só possa ser atribuído durante a declaração ou dentro do construtor da classe. Qualquer tentativa posterior de reatribuição gerará um erro no compilador.

Capítulo 5: O Pilar do Encapsulamento — Controle Severo de Acesso

5.1 Fundamentação Teórica do Encapsulamento

> **Definição Detalhada:** O Encapsulamento é o pilar da POO encarregado de ocultar a complexidade interna de um objeto e proteger o seu estado contra modificações indevidas ou inconsistentes advindas de código externo. Ele estabelece uma fronteira clara entre a interface pública do objeto e seus detalhes de implementação interna.

Através do encapsulamento, garantimos a **Manutenção de Invariantes de Negócio**: um objeto nunca deve entrar em um estado inválido (ex: uma conta bancária com saldo negativo não autorizado ou uma idade menor que zero).

5.2 Modificadores de Acesso do TypeScript

O TypeScript oferece três modificadores primários de visibilidade que atuam no tempo de compilação:

- `public` : O elemento pode ser acessado livremente de qualquer lugar (padrão implícito caso omitido).
- `protected` : O elemento é acessível apenas dentro da própria classe e por suas subclasses derivadas. É invisível para o código externo.
- `private` : O elemento só pode ser acessado internamente dentro do escopo da própria classe que o declarou. Subclasses não possuem acesso direto.

5.3 Demonstração Prática de Proteção de Estado

```
class CofreDigital {
    public dono: string;
    private segredoMestre: string;
    protected codigoEmergencia: number;

    constructor(dono: string, segredo: string, codigo: number) {
        this.dono = dono;
        this.segredoMestre = segredo;
        this.codigoEmergencia = codigo;
    }
}
```

Capítulo 6: Encapsulamento Avançado — Private Nativo vs Modificador TS

6.1 Modificador private do TS vs Private Fields do JavaScript (#)

> **Definição & Diferença Crítica:**
>
> - `private` (TypeScript): É apenas uma checagem em tempo de compilação. Ao ser transpilado para JS antigo, o atributo se torna uma propriedade comum e pode ser acessado via reflexão ou inspeção direta.
> - `#field` (Campos Privados ECMAScript): É um privado real em nível de Runtime (motor V8/Navegador). O atributo é fisicamente inacessível fora da classe, lançando exceção se inspecionado.

```
class SistemaSegurancaAvancado {
    // Privado do TS (Removido na transpilação)
    private tokenEfemero: string;

    // Privado Nativo do JavaScript (Garantido em Runtime)
    #chaveCriptografica: string;

    constructor(token: string, chave: string) {
        this.tokenEfemero = token;
        this.#chaveCriptografica = chave;
    }
}
```

6.2 Métodos Privados e de Suporte Interno

Encapsulamento não se restringe a propriedades! Métodos que realizam etapas intermediárias complexas ou cálculos sensíveis devem ser marcados como `private` ou `protected` para evitar exposição de detalhes de implementação.

Capítulo 7: Invariantes de Negócio com Acessores (Getters e Setters)

7.1 Controle de Estado via Getters e Setters

> **Definição:** Getters e Setters (Properties) são métodos especiais interceptadores sintaticamente acessados como se fossem atributos públicos comuns, mas que executam lógica interna customizada ao ler ou atribuir valores.

```
class TemperaturaSensor {
    private _kelvin: number;

    constructor(kelvinInicial: number) {
        if (kelvinInicial < 0) throw new Error("Zero absoluto violado!");
        this._kelvin = kelvinInicial;
    }

    // Getter em Celsius
    get celsius(): number {
        return this._kelvin - 273.15;
    }

    // Setter em Celsius com garantia de invariante
    set celsius(valor: number) {
        if (valor < -273.15) {
            throw new Error("Temperatura abaixo do zero absoluto!");
        }
        this._kelvin = valor + 273.15;
    }
}
```

7.2 Vantagens do Acesso Controlado

- **Validação Imediata:** Rejeita dados corrompidos no momento da atribuição.
- **Propriedades Somente-Leitura Calculadas:** Permite expor dados derivados dinamicamente (como a conversão de Kelvin para Celsius) sem criar atributos redundantes na memória.

Capítulo 8: O Pilar da Herança — Reutilização e Especialização

8.1 Conceito de Herança Simples (extends)

> **Definição Detalhada:** Herança é o mecanismo de POO através do qual uma classe derivada (filha ou subclasse) herda todos os membros acessíveis (atributos e métodos) de uma classe ancestral (pai ou superclasse), podendo estendê-la com novos comportamentos ou especializá-la.

```
// Classe Pai (Superclasse)
class Conta {
    constructor(
        public readonly numero: string,
        protected saldo: number = 0
    ) {}

    public depositar(valor: number): void {
        if (valor > 0) this.saldo += valor;
    }
}

// Classe Filha (Subclasse Especializada)
class ContaPoupanca extends Conta {
    constructor(numero: string, public taxaJuros: number) {
        super(numero); // Chamada obrigatória ao construtor do Pai!
    }

    public renderJuros(): void {
        this.saldo += this.saldo * this.taxaJuros;
    }
}
```

8.2 A Função da Palavra-Chave super

A instrução `super` executa duas funções vitais no contexto de herança: chamar o construtor da superclasse durante a instanciação da subclasse e invocar métodos herdados que foram sobrescritos.

Capítulo 9: Sobrescrita de Métodos e Segurança com override

9.1 Sobrescrita de Métodos (Method Overriding)

> **Definição:** Ocorre quando uma subclasse redefine a implementação de um método já existente na sua superclasse para adaptar o comportamento à sua necessidade específica.

9.2 Garantia Estática com a Palavra-Chave override

> **Definição:** O modificador `override` (introduzido no TS 4.3) instrui explicitamente o compilador a verificar se o método anotado realmente existe na superclasse. Se o método pai for renomeado ou removido, o TS gera um erro imediato.

```
class ContaCorrente extends Conta {
    private limiteChequeEspecial: number = 1000;

    // Garantia estática de que o método existe no Pai
    override public sacar(valor: number): boolean {
        if (valor <= this.saldo + this.limiteChequeEspecial) {
            this.saldo -= valor;
            return true;
        }
        return false;
    }
}
```

9.3 Princípio da Substituição de Liskov (LSP)

A sobrescrita deve estritamente respeitar o contrato original do pai. Uma subclasse deve ser capaz de substituir a superclasse em qualquer ponto do sistema sem alterar a corretude do programa.

Capítulo 10: O Pilar do Polimorfismo e Despacho Dinâmico

10.1 Fundamentação Teórica do Polimorfismo

> **Definição Detalhada:** Polimorfismo ("muitas formas") é a capacidade de tratar objetos de diferentes classes derivadas através de uma única interface ou tipo base comum. Em tempo de execução, a chamada do método invoca a implementação específica do objeto concreto (Despacho Dinâmico / Dynamic Dispatch).

```
class MeioPagamento {
    public processar(valor: number): void {
        console.log(`Processando pagamento genérico de R$ ${valor}`);
    }
}

class PagamentoPix extends MeioPagamento {
    override public processar(valor: number): void {
        console.log(`Chave Pix gerada. Pagando R$ ${valor} com desconto de 5%`);
    }
}

class PagamentoCartao extends MeioPagamento {
    override public processar(valor: number): void {
        console.log(`Transação de R$ ${valor} enviada à adquirente do Cartão.`);
    }
}
```

10.2 Polimorfismo em Ação (Coleções Heterogêneas)

```
// Trata todos os meios como o tipo abstrato/pai MeioPagamento
const transacoes: MeioPagamento[] = [
    new PagamentoPix(),
    new PagamentoCartao(),
    new PagamentoPix()
];

// O comportamento individual é resolvido dinamicamente em runtime!
transacoes.forEach(t => t.processar(150.00));
```

Capítulo 11: Interfaces — Contratos Puros de Arquitetura

11.1 Conceito e Propósito das Interfaces

> **Definição:** Uma Interface no TypeScript é uma estrutura 100% abstrata que define um contrato puramente sintático. Ela lista propriedades, métodos e suas assinaturas, sem fornecer qualquer código de implementação ou estado.

Interfaces são completamente eliminadas durante o processo de transpilação e não ocupam espaço no arquivo JavaScript final gerado.

11.2 Implementação de Contratos (implements)

```
interface Repositorio<T> {
    salvar(item: T): Promise<boolean>;
    buscarPorId(id: string): Promise<T | null>;
}

class RepositorioUsuarioSQL implements Repositorio<Usuario> {
    async salvar(usuario: Usuario): Promise<boolean> {
        console.log("Executando query INSERT INTO usuarios...");
        return true;
    }

    async buscarPorId(id: string): Promise<Usuario | null> {
        return null;
    }
}
```

Capítulo 12: Composição de Contratos e Múltiplas Interfaces

12.1 Múltiplas Interfaces (Simulação de Herança Múltipla)

> **Definição:** Como linguagens modernas proíbem a herança múltipla de classes para evitar a ambiguidade do "Problema do Diamante", o TypeScript permite que uma classe assine múltiplos contratos independentes separando-os por vírgula no `implements`.

```
interface Autenticavel { autenticar(token: string): boolean; }
interface Auditavel { gerarLog(): string; }

class DiretorExecutivo implements Autenticavel, Auditavel {
    autenticar(token: string): boolean { return token === "VIP-999"; }
    gerarLog(): string { return `[AUDITORIA]: Acesso em ${new Date()}`; }
}
```

12.2 Extensão de Interfaces (extends entre Interfaces)

Assim como classes, interfaces podem estender uma ou múltiplas interfaces anteriores para formar contratos compostos mais ricos.

```
interface Leitor { ler(): string; }
interface Escritor { escrever(dados: string): void; }

// Interface composta por herança dupla
interface LeitorEscritor extends Leitor, Escritor {}
```

Capítulo 13: Classes Abstratas vs Interfaces — A Decisão Arquitetural

13.1 Conceito de Classe Abstrata (abstract class)

> **Definição Detalhada:** Uma Classe Abstrata ocupa uma posição intermediária entre uma Interface e uma Classe Concreta. Ela pode conter membros totalmente implementados (estado e código reaproveitável) e métodos abstratos sem corpo que forçam a implementação nas subclasses.

```
abstract class ProcessadorDocumento {
    // Método Concreto Reutilizável (Template Method Pattern)
    public processarArquivo(caminho: string): void {
        this.abrirArquivo(caminho);
        this.extrairDados();
        this.fecharArquivo();
    }

    protected abrirArquivo(c: string): void { console.log(`Abrindo: ${c}`); }
    protected fecharArquivo(): void { console.log("Fechando arquivo."); }

    // Método abstrato obriga a especialização no filho
    abstract protected extrairDados(): void;
}
```

13.2 Tabela Comparativa: Interface vs Classe Abstrata

- **Interface:** 0% de implementação, compilação limpa (sem JS), suporta múltipla implementação.
- **Classe Abstrata:** Implementação parcial (0% a 100%), gera código JS na transpilação, restrita à herança única.

Capítulo 14: Union Types, Type Narrowing e Discriminated Unions

14.1 Union Types (|) e Estreitamento de Tipos (Narrowing)

> **Definição:** Union Types permitem que um parâmetro ou variável aceite múltiplos tipos alternativos. O *Type Narrowing* é a capacidade do compilador de deduzir o tipo exato dentro de um bloco condicional através de verificações como `typeof`, `instanceof` ou operadores `in`.

```
function formatarEntrada(valor: string | number | Date): string {
    if (typeof valor === "string") return valor.trim().toUpperCase();
    if (typeof valor === "number") return `R$ ${valor.toFixed(2)}`;
    return valor.toISOString(); // O TS sabe que aqui é estritamente Date!
}
```

14.2 Discriminated Unions (Uniões Discriminadas)

> **Definição:** Padrão de arquitetura no qual múltiplos tipos possuem uma propriedade literal em comum (ex: `kind` ou `type`) usada como discriminador para pattern matching seguro.

```
type EventoSucesso = { kind: "SUCCESS"; payload: string };
type EventoErro = { kind: "ERROR"; erro: Error };
type EventoRede = EventoSucesso | EventoErro;
```

Capítulo 15: Programação Genérica (Generics) — Fundamentos

15.1 O que são Generics (<T>)

> **Definição Detalhada:** Generics são variáveis de tipo. Eles permitem projetar classes, interfaces e funções parametrizadas capazes de operar sobre múltiplos tipos mantendo a integridade estática completa sem recorrer ao `any`.

```
// Sem Generics: Perda do tipo exato (Retorna any ou exige cast)
// Com Generics: Captura dinâmica do tipo passado
class RespostaEnvelope<TData> {
    constructor(
        public sucesso: boolean,
        public dados: TData,
        public timestamp: number = Date.now()
    ) {}
}

// Instanciação fortemente tipada
const respString = new RespostaEnvelope<string>(true, "Sucesso");
const respNum = new RespostaEnvelope<number[]>(true, [1, 2, 3]);
```

15.2 Múltiplos Parâmetros de Tipo

É possível utilizar múltiplos parâmetros genéricos nomeados convencionalmente como `T`, `U`, `K`, `V`.

```
class Par<TK, TV> {
    constructor(public chave: TK, public valor: TV) {}
}
```

Capítulo 16: Restrições Genéricas (Generic Constraints)

16.1 O Operador extends em Generics

> **Definição:** Por padrão, um parâmetro genérico `<T>` aceita absolutamente qualquer tipo. A restrição genérica (`T extends Interface`) delimita que o tipo passado obrigatoriamente possua determinada estrutura ou herde de determinada classe.

```
interface TemComprimento { length: number; }

// Aceita apenas tipos que possuam a propriedade length
function imprimirComprimento<T extends TemComprimento>(item: T): void {
    console.log(`Tamanho: ${item.length}`);
}

imprimirComprimento("Texto em TypeScript"); // Funciona (String tem length)
imprimirComprimento([10, 20, 30, 40]);       // Funciona (Array tem length)
// imprimirComprimento(12345); // Erro! Number não tem propriedade length.
```

16.2 Uso do Operador keyof com Generics

> **Definição:** O operador `keyof` extrai a união de todas as chaves públicas de um tipo de objeto, permitindo garantir estritamente o acesso a propriedades válidas.

```
function obterPropriedade<T, K extends keyof T>(obj: T, chave: K): T[K] {
    return obj[chave];
}
```

Capítulo 17: Utilitários de Tipo do TypeScript (Utility Types I)

17.1 Transformações de Tipos Integradas

O TypeScript oferece utilitários genéricos globais para facilitação de metamodelagem de tipos:

- `Partial<T>` : Converte todas as propriedades do tipo `T` em opcionais (`?`).
- `Required<T>` : Torna todas as propriedades obrigatoriamente preenchidas (remove `?`).
- `Readonly<T>` : Marca todas as propriedades como imutáveis.

```
interface PerfilUsuario {
    id: string;
    nome: string;
    bio?: string;
}

// Útil para DTOs de atualização parcial (PATCH)
type AtualizarPerfilDTO = Partial<PerfilUsuario>;

// Garante imutabilidade absoluta dos dados recuperados
type PerfilCongelado = Readonly<PerfilUsuario>;
```

17.2 Modelagem de DTOs e Contratos de API

A utilização de `Partial<T>` evita a repetição desnecessária de interfaces separadas para criação e edição de entidades.

Capítulo 18: Utilitários de Tipo do TypeScript (Utility Types II)

18.1 Seleção e Filtragem com Pick e Omit

> **Definições:**
> - `Pick<T, K>` : Cria um novo tipo extraindo apenas as chaves especificadas `K` contidas em `T` .
> - `Omit<T, K>` : Cria um novo tipo removendo as chaves especificadas `K` do tipo `T` .

```
interface Artigo {
    id: string;
    titulo: string;
    conteudo: string;
    autorId: string;
    rascunho: boolean;
}

// Seleciona apenas id e titulo para exibição em lista resumida
type ArtigoResumoDTO = Pick<Artigo, "id" | "titulo">;

// Omite propriedades geradas pelo servidor para formulário de criação
type CriarArtigoDTO = Omit<Artigo, "id" | "rascunho">;
```

18.2 Mapeamento Dinâmico com Record<K, T>

> **Definição:** Constrói um tipo de objeto cujas chaves são do tipo `K` e os valores são do tipo `T` .

```
type Permissoes = "LEITURA" | "ESCRITA" | "EXCLUSAO";
type MapaAcesso = Record<Permissoes, boolean>;
```

Capítulo 19: Projeto de Arquitetura Integrada — Padrões em Ação

19.1 Unindo Encapsulamento, Herança, Interfaces e Generics

O exemplo abaixo demonstra a convergência de todos os pilares em uma arquitetura de serviço desacoplada e testável:

```
// Contrato com Generics e Interfaces
interface Entidade { id: string; }

abstract class ServicoBase<T extends Entidade> {
    private registros: Map<string, T> = new Map();

    public salvar(item: T): void {
        this.validarInvariantes(item);
        this.registros.set(item.id, item);
    }

    public buscar(id: string): T | undefined {
        return this.registros.get(id);
    }

    // Hook abstrato forçado para validação polimórfica
    abstract protected validarInvariantes(item: T): void;
}
```

19.2 Especialização de Serviços Concretos

```
interface ClienteEntidade extends Entidade { email: string; }

class ServicoCliente extends ServicoBase<ClienteEntidade> {
    override protected validarInvariantes(cliente: ClienteEntidade): void {
        if (!cliente.email.includes("@")) {
            throw new Error("Email inválido no cadastro do cliente!");
        }
    }
}
```

Capítulo 20: Checklist de Boas Práticas e Conclusão

20.1 As 10 Regras de Ouro da Arquitetura TypeScript

1. **Ative Sempre o Modo Estrito:** Defina `"strict": true` no arquivo `tsconfig.json`.
2. **Evite o Tipo any:** Dê preferência a `unknown` para entradas dinâmicas incertas.
3. **Ganta o Encapsulamento Real:** Proteja o estado interno usando `private` ou `#fields` nativos e exponha alterações somente via métodos/setters com validação.
4. **Aproveite a Tipagem Estrutural:** Prefira `Interfaces` para objetos de domínio extensíveis e `Type Aliases` para uniões e tuplas.
5. **Use override Explicito:** Sempre anote métodos sobrescritos para proteção contra mudanças na superclasse.
6. **Favoreça a Composição à Herança:** Prefira pequenas interfaces focadas (Single Responsibility Principle) a hierarquias profundas de herança.
7. **Trate Null e Undefined com Rigor:** Mantenha `strictNullChecks` ligado para eliminar exceções de ponteiro nulo.
8. **Explore Discriminated Unions:** Garanta checagem exaustiva em instruções do tipo `switch`.
9. **Crie Tipos Genéricos Restritos:** Use `extends` para limitar o escopo dos parâmetros em Generics.
10. **Aproveite Utility Types:** Torne seu código DRY (Don't Repeat Yourself) utilizando `Pick`, `Omit` e `Partial`.

20.2 Conclusão do Manual

O domínio do TypeScript com Programação Orientada a Objetos transforma o desenvolvimento de softwares, elevando a manutenibilidade, previsibilidade e confiabilidade de sistemas de grande escala.